

Unidad III: Complejidad Computacional

Las clases P y EXP

Clase 15 - Lógica para Ciencia de la Computación/Teoría de la Computación

Prof. Miguel Romero

- ¿Qué problemas pueden ser resueltos eficientemente?

Recordatorio: Máquinas de Turing (deterministas)

Definición:

Una **máquina de Turing (determinista)** (MT o MTD) es una tupla

$M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$, donde:

- Q es un conjunto finito de **estados**.
- Σ es el **alfabeto de entrada**.
- Γ es el **alfabeto de la cinta**, donde $\Sigma \subseteq \Gamma$, y $\vdash, B \in \Gamma \setminus \Sigma$.
- q_0 es el **estado inicial**.
- q_f es el **estado final**.
- $\delta : (Q \setminus \{q_f\}) \times \Gamma \rightarrow Q \times \Gamma \times \{\triangleleft, \triangleright\}$ es una función parcial llamada la **función de transición**.

Recordatorio: configuraciones

Una **configuración** de una MT está dada por:

- El **estado actual** de la MT.
- La **posición actual** de la cabeza lectora.
- El **contenido** de la cinta (ignorando B irrelevantes hacia la derecha).

Definición:

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ una MT. Una **configuración** de M es representada por una palabra de la forma uqv , con $u, v \in \Gamma^*$ y $q \in Q$, donde:

- q es el estado actual de M .
- $|u|$ es la posición actual de la cabeza lectora.
- uv es el contenido de la cinta.

Recordatorio: tipos de configuraciones

Definición:

Decimos que una configuración uqv es:

- de **aceptación** si $q = q_f$.
- de **detención** si $\delta(q, a)$ no está definido, donde $v = av'$.
- de **rechazo** si es de detención y $q \neq q_f$.

Recordatorio: siguiente configuración

Definición:

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ una MT. Se define la relación **siguiente configuración** $\xrightarrow{M}$ entre configuraciones de M como:

$C \xrightarrow{M} C'$ si y sólo si

- Existe transición $\delta(q, a) = (q', b, \triangleleft)$, $u, v \in \Gamma^*$ y $d \in \Gamma$ tal que:

$$C = u d \textcolor{brown}{q} a v \qquad C' = u \textcolor{brown}{q}' d b v$$

- O existe transición $\delta(q, a) = (q', b, \triangleright)$ y $u, v \in \Gamma^*$ tal que:

$$C = u \textcolor{brown}{q} a v \qquad C' = u b \textcolor{brown}{q}' v$$

Recordatorio: ejecuciones

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ una MT y $w \in \Sigma^*$.

Definición:

- Una **ejecución** ρ de M es una secuencia de configuraciones
$$\rho = C_0, C_1, C_2, \dots \text{ (no necesariamente finita)}$$
tal que $C_i \xrightarrow{M} C_{i+1}$ para todo $i \geq 0$.
- Decimos que $\rho = C_0, C_1, C_2, \dots$ es **la ejecución** de M sobre w si:
 - ρ es una ejecución de M .
 - $C_0 = \vdash q_0 w$. (configuración inicial de M sobre w)
 - Si $\rho = C_0, \dots, C_m$ es **finito**, entonces C_m es de **detención**.
- En caso que ρ sea **finito**, decimos que M se **detiene** con w .

Recordatorio: lenguaje aceptado

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ una MT y $w \in \Sigma^*$.

Definición:

- M **acepta** w si la ejecución ρ de M sobre w cumple que:
 - $\rho = C_0, \dots, C_m$ es finita. (M se detiene con w)
 - C_m es una configuración de aceptación.
- El **lenguaje aceptado por** M se define como:

$$L(M) = \{w \in \Sigma^* \mid M \text{ acepta } w\}.$$

Recordatorio: clases R, RE, co-RE

Clases de lenguajes vistas hasta el momento:

RE = clase de los lenguajes **recursivamente enumerables**.

- L es recursivamente enumerable si existe MT M tal que $L = L(M)$.

co-RE = clase de los lenguajes cuyo **complemento** es recursivamente enumerable.

R = clase de los lenguajes **decidibles**.

- L es decidable si existe MT M que se detiene con toda entrada, y $L = L(M)$.

Tiempo de ejecución de una MT

Sea una MT $M = (Q, \Sigma, \Gamma, q_0, q_f, \delta)$ que se detiene con toda entrada.

Definición:

- Sea una palabra $w \in \Sigma^*$ y $C_0, \dots, C_m$ la ejecución de M sobre w . Definimos la **cantidad de pasos** $p_M(w)$ de M sobre w como m .
- El **tiempo de ejecución (de peor caso)** de M es $t_M : \mathbb{N} \rightarrow \mathbb{N}$ tal que:

$$t_M(n) = \max\{p_M(w) \mid w \in \Sigma^* \text{ y } |w| = n\}.$$

Comentarios:

- A t_M también se le llama **complejidad tiempo**.
- Decimos que M **toma tiempo** t_M .

Definición:

Sean dos funciones $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Decimos que f es $O(g(n))$, denotado como $f = O(g(n))$, si existe un natural $n_0 \geq 0$ y un real $c \geq 0$ tal que:

$$f(n) \leq c \cdot g(n) \quad \text{para todo } n \geq n_0.$$

Idea:

" $f \leq g$ " = f es **menor o igual** que g (en el sentido asintótico).

Ejemplos:

- Sea $f(n) = 5n^3 + 2n^2 + 22n + 6$. Entonces $f(n) = O(n^3)$.
- Sea $f(n) = 2^n + 1000n^{100}$. Entonces $f(n) = O(2^n)$.
- Sea $f(n) = 2^n n^2 + 2^n + n^2$. Entonces $f(n) = O(2^n n^2)$.
- Sea $f(n) = \log_2 n + n$. Entonces $f(n) = O(n)$.
- Sea $f(n) = \log_2 n + n^{0.001}$. Entonces $f(n) = O(n^{0.001})$.

Receta:

- Dejar el término de mayor orden e ignorar constantes.
- El producto de funciones es mayor que sus factores.
- Relaciones útiles ($0 < \varepsilon \leq 1$, $k \geq 1$, $d \geq 1$):

$$1 \leq \log_2 n \leq n^\varepsilon \leq n \leq n^k \leq 2^n \leq 2^{dn} \leq 2^{n^d}$$

Ejemplo

Considere el siguiente lenguaje L y el siguiente algoritmo M que lo decide:

$$L = \{0^k 1^k \mid k \geq 0\}.$$

Sobre entrada $w \in \{0, 1\}^*$:

1. Mover la cabeza hacia la derecha hasta el primer blanco.
Si hay un 1 antes de un 0, **rechazar**.
2. Repetir mientras quede un 0 y un 1:
 - Mover la cabeza hacia la derecha desde el inicio.
Tachar el primer 0 y el primer 1 que se encuentre.
3. Si no quedan 0's ni 1's, **aceptar**. En caso contrario, **rechazar**.

¿Cuánto es el tiempo de ejecución de M en términos de la notación O ?

Ejemplo

Sobre entrada $w \in \{0,1\}^*$:

1. Mover la cabeza hacia la derecha hasta el primer blanco.
Si hay un 1 antes de un 0, **rechazar**.
2. Repetir mientras quede un 0 y un 1:
 - Mover la cabeza hacia la derecha desde el inicio.
Tachar el primer 0 y el primer 1 que se encuentre.
3. Si no quedan 0's ni 1's, **aceptar**. En caso contrario, **rechazar**.

Análisis de M :

- ¿Cuánto toma el paso (1)? $O(n)$.
- ¿Cuánto toma el ciclo while del paso (2)?
 - ¿Cuánto toma cada iteración del ciclo while? $O(n)$.
 - ¿Cuántas iteraciones hay? $O(n)$.
 - En total el ciclo while toma $O(n^2)$.
- ¿Cuánto toma el paso (3)? $O(n)$.

El tiempo de ejecución de M es $t_M(n) = O(n) + O(n^2) + O(n) = O(n^2)$.

Complejidad de un lenguaje

Definición:

Sea una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. Un lenguaje L se puede resolver (o decidir) en tiempo $O(f(n))$, si existe una MT M que se detiene con toda entrada tal que $L = L(M)$ y $t_M(n) = O(f(n))$.

Comentarios:

- El ejemplo anterior L puede ser resuelto en tiempo $O(n^2)$.

La clase $\text{DTIME}(f(n))$

Definición:

Sea una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. La clase de complejidad $\text{DTIME}(f(n))$ es la clase de todos los lenguajes L que pueden ser resueltos en tiempo $O(f(n))$.

Comentarios:

- El ejemplo anterior $L \in \text{DTIME}(n^2)$.
- Si $f(n) = O(g(n))$, entonces $\text{DTIME}(f(n)) \subseteq \text{DTIME}(g(n))$.
- Las clases DTIME se definen con el modelo básico de MTs (una cinta).

¿Qué pasa si tomamos otro modelo determinista de MTs?

Ejemplo

Considere el siguiente algoritmo M con 2 cintas para L :

$$L = \{0^k 1^k \mid k \geq 0\}.$$

Sobre entrada $w \in \{0, 1\}^*$:

1. En **cinta 1**, mover cabeza a la derecha hasta el primer blanco.
Si hay un 1 antes de un 0, **rechazar**.
2. En **cinta 1**, mover cabeza a la derecha desde el inicio hasta el primer 1.
Copiar cada 0 visto a la **cinta 2**.
3. En **cinta 1**, seguir moviendo cabeza a la derecha hasta el primer blanco.
Por cada 1 visto en la **cinta 1**, tachar un 0 en la **cinta 2**.
Si no hay suficientes 0's, **rechazar**.
4. Si no quedan 0's en la **cinta 2**, **aceptar**. En caso contrario, **rechazar**.

¿Cuánto es el tiempo de ejecución de M en términos de la notación O ?

Ejemplo

Sobre entrada $w \in \{0,1\}^*$:

1. En **cinta 1**, mover cabeza a la derecha hasta el primer blanco.
Si hay un 1 antes de un 0, **rechazar**.
2. En **cinta 1**, mover cabeza a la derecha desde el inicio hasta el primer 1.
Copiar cada 0 visto a la **cinta 2**.
3. En **cinta 1**, seguir moviendo cabeza a la derecha hasta el primer blanco.
Por cada 1 visto en la **cinta 1**, tachar un 0 en la **cinta 2**.
Si no hay suficientes 0's, **rechazar**.
4. Si no quedan 0's en la **cinta 2**, **aceptar**. En caso contrario, **rechazar**.

Análisis de M :

- El paso (1) toma tiempo $O(n)$.
- El paso (2)-(3) toma tiempo $O(n)$.
- El paso (4) toma tiempo $O(n)$.

El tiempo de ejecución de M es $t_M(n) = O(n) + O(n) + O(n) = O(n)$.

Ejemplo

Consecuencia:

- ¿Puede L ser resuelto en tiempo $O(n)$?
 - Si consideremos MTs con múltiples cinta, **sí!**
 - Si consideremos MTs con una cinta, **no!** (se puede demostrar.)
- La definición de DTIME depende del modelo de MT.

¿Cómo definimos clases de complejidad más robustas?

Una cinta vs múltiples cintas

Teorema:

Sea $t : \mathbb{N} \rightarrow \mathbb{N}$ tal que $t(n) \geq n$. Toda MT con múltiples cintas que toma tiempo $t(n)$, puede ser simulada por una MT de una cinta que toma tiempo $O(t^2(n))$.

Demostración:

Propuesta. (revisar la simulación de múltiples a una cinta.)

Comentarios:

- Hay sólo una **diferencia polinomial** entre estos dos modelos.
- Se tienen resultados similares para modelos deterministas conocidos.
(cinta doblemente infinita, múltiples cabezas, cinta bi-dimensional, modelo RAM,...)

La clase P

Definimos la clase de complejidad P como sigue:

$$P = \bigcup_{k \in \mathbb{N}} \text{DTIME}(n^k)$$

Comentarios:

- $L \in P \iff L$ se puede resolver en tiempo $O(n^k)$, para algún $k \in \mathbb{N}$.
- Decimos que L se puede resolver en **tiempo polinomial**.
- La definición de P **no** depende del modelo escogido de MTD.
- P formaliza los problemas que pueden ser resueltos **eficientemente**.

¿Es la clase P una buena formalización de eficiencia?

La clase EXP

Definimos la clase de complejidad P como sigue:

$$\text{EXP} = \bigcup_{k \in \mathbb{N}} \text{DTIME}(2^{n^k})$$

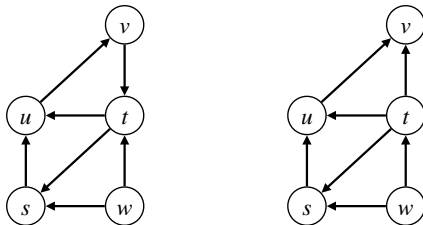
Comentarios:

- $L \in \text{EXP} \iff L$ se puede resolver en tiempo $O(2^{n^k})$, para algún $k \in \mathbb{N}$.
- Decimos que L se puede resolver en **tiempo exponencial**.
- La definición de EXP **no** depende del modelo escogido de MTD.

Problemas en P

Considere el siguiente problema:

$\text{PATH} = \{ \langle G, s, t \rangle \mid G \text{ grafo dirigido que tiene un camino dirigido de } s \text{ a } t \}$.



Veamos que $\text{PATH} \in \text{P}$.

Algunas convenciones

Para analizar problemas concretos, tomaremos las siguientes convenciones:

- Escribiremos **algoritmos de alto nivel** sin hacer referencia a MTs.
- ¿Cómo analizamos el tiempo de ejecución de estos algoritmos?
 - Estimamos la cantidad de “**pasos**” (peor caso y notación O).
 - La estimación se hace con respecto a un **parámetro representativo**, y **no** con respecto al largo de la codificación de la entrada.

Algunas convenciones

Ejemplos:

Para PATH, podemos tomar la siguiente codificación sobre $\{0, 1, \#\}$:

$$\langle G, s, t \rangle = 0^m \# \langle A_G \rangle \# 0^i \# 0^j$$

donde

- m es la cantidad de nodos de G .
- Asignamos a cada nodo, un número en $\{1, \dots, m\}$.
- A_G es la **matriz de adyacencia** de G y $\langle A_G \rangle$ su codificación.
- s es el i -ésimo nodo y t el j -ésimo.

Sea $n = |\langle G, s, t \rangle|$. Podemos tomar m como parámetro:

$$m = O(n) \quad n = O(m^2)$$

Obtenemos que n y m tienen sólo una diferencia polinomial.

(esto se cumple para cualquier codificación razonable de (G, s, t) .)

Aplicaremos lo mismo para problemas de grafos en general.

Algunas convenciones

Ejemplos:

Podemos codificar fórmulas proposicionales φ sobre $\{\neg, \wedge, \vee, \rightarrow, \leftarrow, (,), 0, 1\}$:

- Si φ tiene ℓ variables, asignamos a cada variable, un número en $\{0, \dots, \ell - 1\}$.
- Escribimos cada variable en binario.

- Ejemplo:

$$\varphi = (x_0 \vee \neg x_1 \vee x_2) \wedge (\neg x_1 \vee x_3 \vee \neg x_4) \wedge (\neg x_0 \vee \neg x_5 \vee \neg x_6 \vee x_7 \vee \neg x_3)$$

$$\langle \varphi \rangle = (0 \vee \neg 1 \vee 10) \wedge (\neg 1 \vee 11 \vee \neg 100) \wedge (\neg 0 \vee \neg 101 \vee \neg 110 \vee 111 \vee \neg 11)$$

Sea $n = |\langle \varphi \rangle|$. Podemos tomar el **largo de φ** , denotado $|\varphi|$, como parámetro:

$|\varphi|$ = cantidad de ocurrencias de conectivos, variables y paréntesis.

Se cumple: $|\varphi| = O(n)$ $n = O(|\varphi| \cdot \log_2 |\varphi|)$

Obtenemos que n y $|\varphi|$ tienen sólo una diferencia polinomial.

(esto se cumple para cualquier codificación razonable de φ .)

Algunas convenciones

Ejemplos:

En general, codificamos números naturales n en **binario**.

Podemos tomar $\log_2 n$ como parámetro:

Obtenemos que $|\langle n \rangle|$ y $\log_2 n$ tienen sólo una diferencia polinomial.

Algunas convenciones

Para analizar problemas concretos, tomaremos las siguientes convenciones:

- Escribiremos algoritmos de alto nivel sin hacer referencia a MTs.
- ¿Cómo analizamos el tiempo de ejecución de estos algoritmos?
 - Estimamos la cantidad de “pasos” (peor caso y notación O).
 - La estimación se hace con respecto a un parámetro representativo, y **no** con respecto al largo de la codificación de la entrada.

Consecuencia:

Un problema está en P si es resuelto por un algoritmo que toma una cantidad de “pasos” polinomial en el parámetro representativo.

(similar para EXP.)

$\text{PATH} = \{ \langle G, s, t \rangle \mid G \text{ grafo dirigido que tiene un camino dirigido de } s \text{ a } t \}.$

Algoritmo M para PATH:

Para entrada (G, s, t) donde G es un grafo dirigido y s, t son nodos:

1. Marcar nodo s .
2. Repetir hasta que ningún nodo nuevo sea marcado:
 - Iterar por todos los arcos (a, b) de G .
Si a está marcado, y b no está marcado, entonces marcar b .
3. Si t está marcado, **aceptar**. En caso contrario, **rechazar**.

¿Cuánto es el tiempo de ejecución de M con respecto a $m =$ cantidad nodos G ?

$$O(1) + m \cdot O(m^2) + O(1) = O(m^3).$$

Luego $\text{PATH} \in \text{P}$.

$SAT = \{ \langle \varphi \rangle \mid \varphi \text{ fórmula proposicional satisfacible} \}.$

Algoritmo M para SAT:

Para entrada φ :

1. Iterar por todas las posibles valuaciones σ para φ :
 - Si $\sigma(\varphi) = 1$, **aceptar**.
2. **Rechazar**.

¿Cuánto es el tiempo de ejecución de M con respecto a $|\varphi|$?

$$2^{|\varphi|} \cdot O(|\varphi|) = O(2^{|\varphi|} \cdot |\varphi|).$$

Luego $SAT \in EXP$.

$$\text{COMPOSITE} = \{ \langle n \rangle \mid n = p \cdot q, \text{ para } p, q > 1 \}.$$

Algoritmo M para COMPOSITE:

Para entrada n :

1. Iterar por todos los posibles números $1 < p < n$:
 - Si p divide a n , **aceptar**.
2. **Rechazar**.

¿Cuánto es el tiempo de ejecución de M con respecto a $\log_2 n$?

$$(n-2) \cdot O(n) = O(n^2) = O(2^{2 \log_2 n}).$$

Luego $\text{COMPOSITE} \in \text{EXP}$.